

Django MVT y ORM: De la Interfaz a MySQL

Guía paso a paso para automatizar datos con Modelos y Formularios.

Gemini Notebook

El Objetivo: Flujo de Datos **Automatizado**



El usuario envía información a través de una interfaz web.

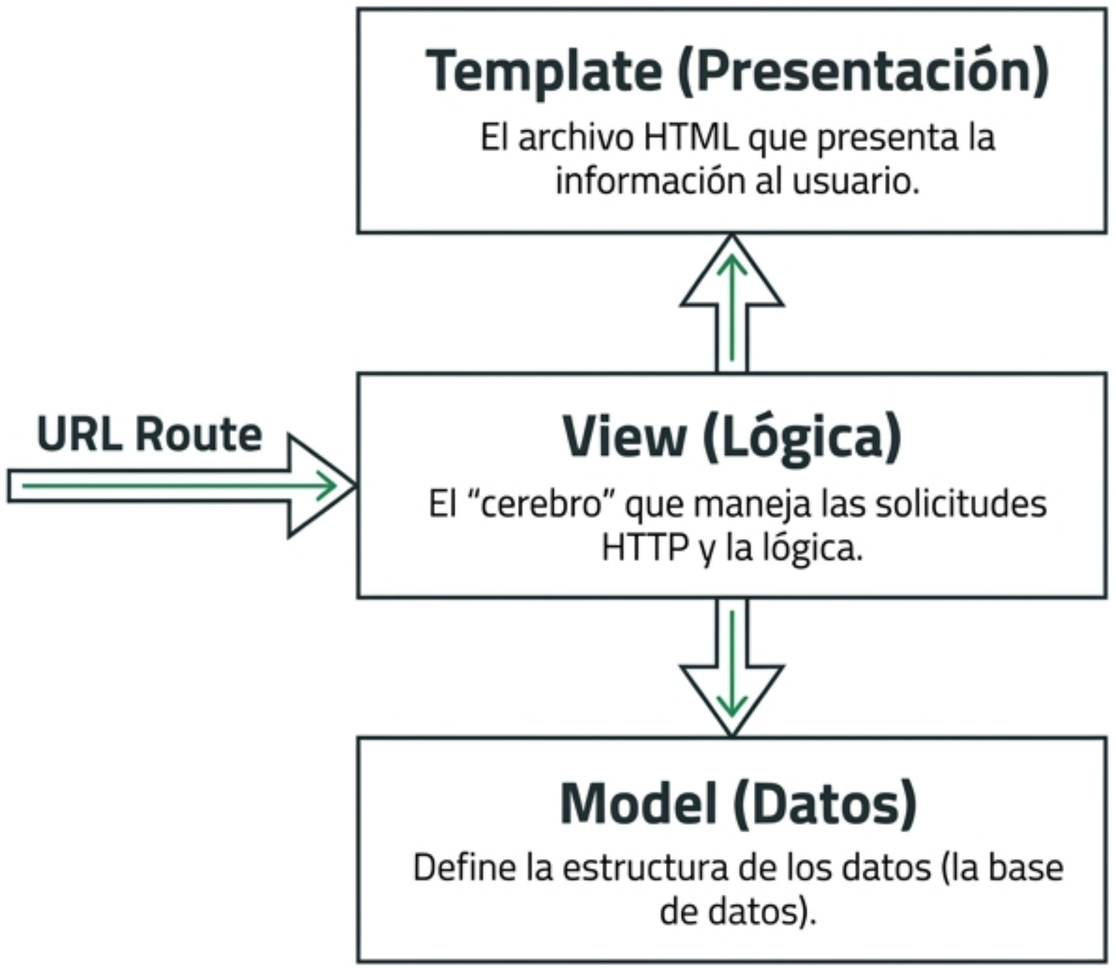
Django procesa, valida y traduce la información.

Los datos se almacenan de forma permanente y segura.

Gemini Notebook

La Arquitectura MVT

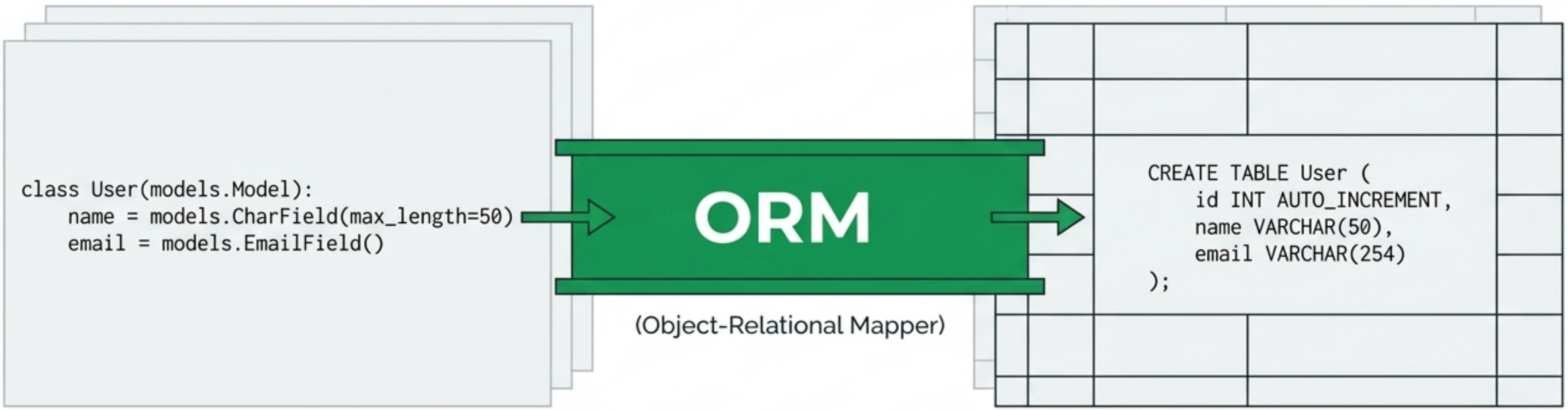
Django separa las responsabilidades en tres capas independientes:



Gemini Notebook

¿Qué es el ORM de Django?

El ORM es un traductor universal. Permite escribir código en Python orientado a objetos y lo convierte automáticamente en consultas SQL complejas para interactuar con la base de datos sin escribir SQL puro.



Gemini Notebook

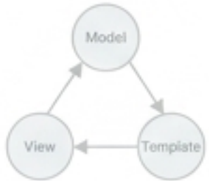
Preparando el Terreno: SQLite vs. MySQL

Django incluye SQLite por defecto, pero para aplicaciones web reales con alta disponibilidad y múltiples usuarios escribiendo datos, MySQL es el estándar requerido.

	SQLite	MySQL
Entorno	Desarrollo y Pruebas locales	Producción web
Concurrencia	Baja (1 usuario a la vez)	Alta (Múltiples usuarios simultáneos)
Tipo de Proyecto	Prototipos pequeños	Aplicaciones web, CMS, eCommerce

Gemini Notebook

Paso 1: Conectar MySQL a Django



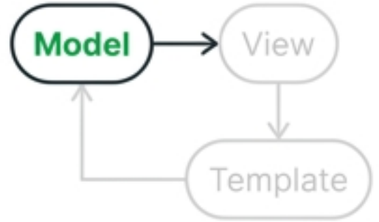
En el archivo settings.py, reemplazamos la configuración predeterminada para apuntar al servidor MySQL.

```
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.mysql',
    'NAME': 'mi_base_datos',
    'USER': 'root',
    'PASSWORD': 'mi_password',
  }
}
```

Motor de conexión MySQL

Credenciales de acceso

Gemini Notebook



Paso 2: Crear el Modelo Class

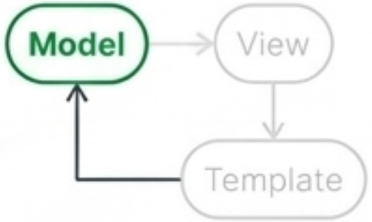
Los modelos en models.py son la única fuente de la verdad de tus datos. Cada atributo de la clase representa una columna en la base de datos.

```
class Task(models.Model):
    title = models.CharField(max_length=100)
    description = models.TextField()
    completed = models.BooleanField(default=False)
```

Se convierte en VARCHAR en MySQL

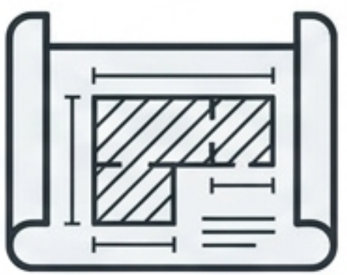
Gemini Notebook

Paso 3: Migraciones - Diseñando el Plano



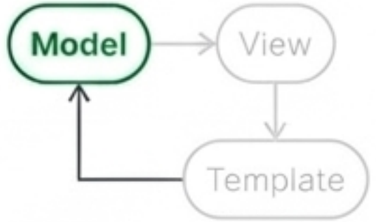
El comando makemigrations escanea tus modelos de Python y genera un archivo de instrucciones (migración). Es el 'plano arquitectónico' que le dice a Django cómo modificar el esquema de la base de datos.

```
$ python manage.py makemigrations
Migrations for 'myapp':
  myapp/migrations/0001_initial.py
    - Create model Task
```



Plano Generado
(No ejecutado aún)

Gemini Notebook



Paso 4: Ejecutar la Migración

El comando `migrate` toma el plano generado y ejecuta las sentencias SQL reales (Data Definition Language) en MySQL para crear las tablas físicas.

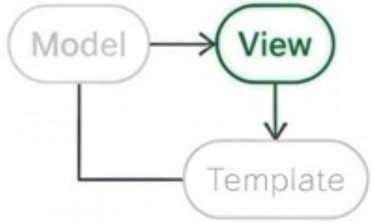
```
$ python manage.py migrate
Operations to perform:
  Apply all migrations: myapp
Running migrations:
  Applying myapp.0001_initial... OK
```

Genera tabla física

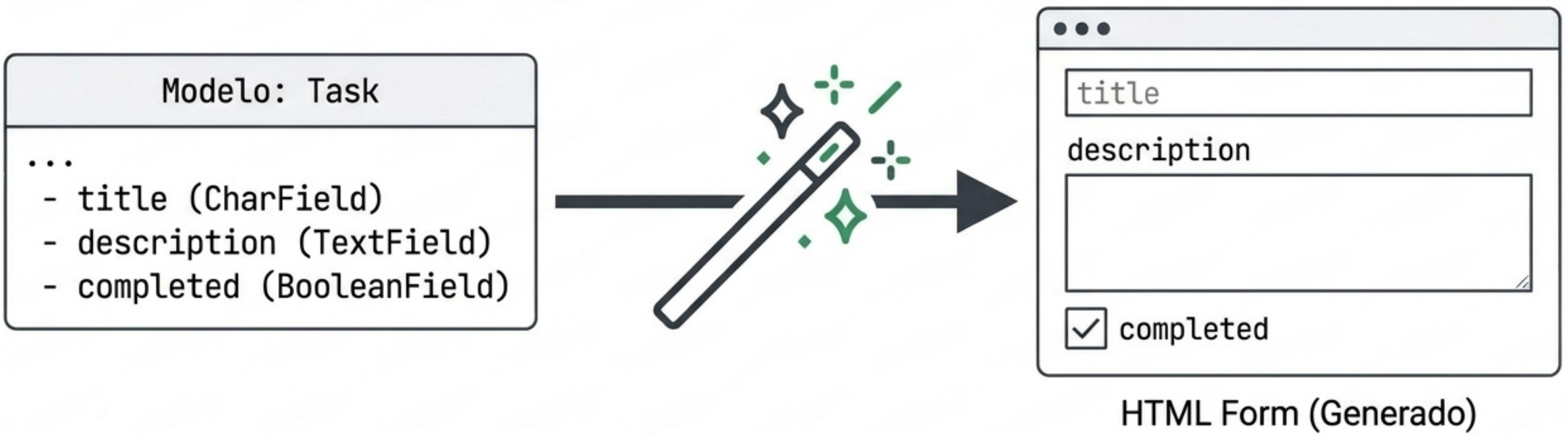
app_task			
id	title	description	completed

Gemini Notebook

Paso 5: Automatización con Formularios



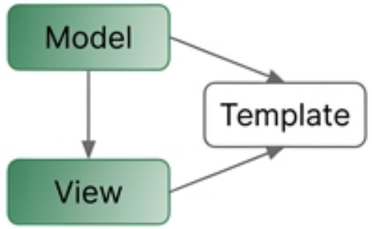
El principio DRY (Don't Repeat Yourself) de Django brilla aquí. En lugar de escribir formularios HTML desde cero, usamos ModelForm para generar automáticamente la interfaz de entrada basándonos exactamente en las reglas de nuestro Modelo.



HTML Form (Generado)

Gemini Notebook

El Código del Formulario



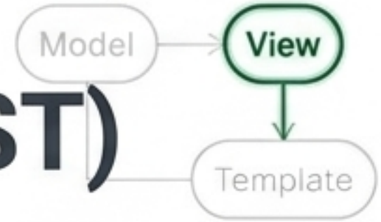
A través de la clase **Meta**, vinculamos directamente el **formulario** al modelo **Task**. Django detecta automáticamente los tipos de datos y crea los campos HTML correctos (texto, área de texto, casilla de verificación).

```
from django import forms
from .models import Task

class TaskForm(forms.ModelForm):
    class Meta:
        model = Task
        fields = ['title','description','completed']
```

El puente automático entre tu Modelo y el HTML.

Gemini Notebook

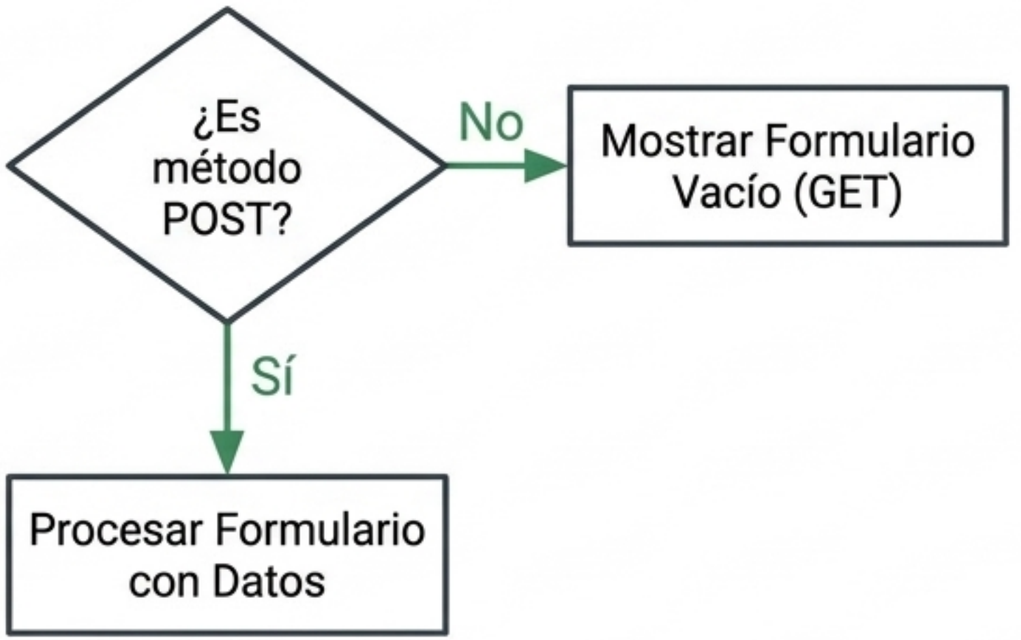


Paso 6: La Lógica en la Vista (GET vs POST)

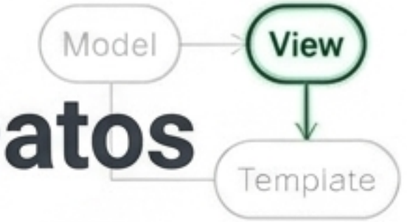
En **views.py**, creamos la función que manejará la interacción del usuario. Diferenciamos entre cargar la página por primera vez (GET) y enviar datos (POST).

```
def task_list(request):

    if request.method == 'POST':
        form = TaskForm(request.POST)
        # Lógica de guardado...
    else:
        form = TaskForm()
```



Gemini Notebook



Paso 7: Validación y Guardado en Base de Datos

Django valida los datos automáticamente con **is_valid()**. Si todo es correcto, un simple **form.save()** instruye al ORM para generar un comando **INSERT de SQL y guardar el registro en MySQL**.

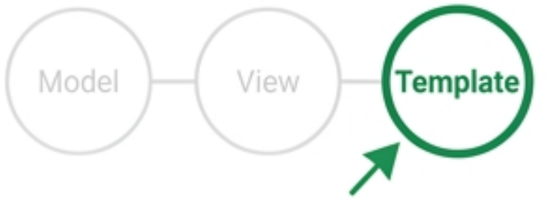
```
if form.is_valid():
    form.save()  # Guarda en MySQL
    return redirect('task_list')
```

Genera:
INSERT INTO...



Gemini Notebook

Paso 8: El Contexto hacia el Template

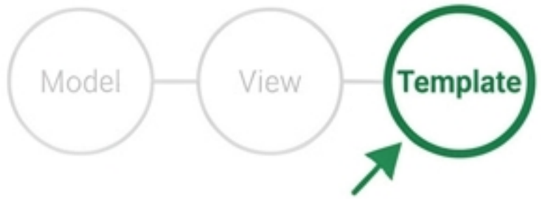


La **Vista** necesita comunicarse con la interfaz de usuario. Para ello, **renderiza** el **Template HTML** y le pasa un diccionario de contexto, inyectando nuestro **TaskForm** directamente en la página web.



Gemini Notebook

El Código del Template



Con el lenguaje de templates de Django, podemos renderizar el formulario completo con una sola variable. El **token CSRF** protege la base de datos contra ataques de falsificación de solicitudes.

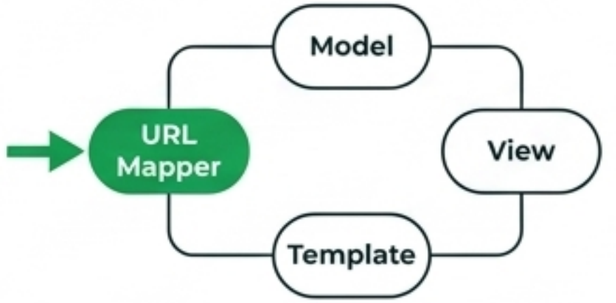
```
<form method='POST'>
    {% csrf_token %}
    {{ form.as_p }}
    <button type='submit'>Guardar</button>
</form>
```

Seguridad anti-hackers integrada

Renderiza todos los inputs HTML automáticamente

Gemini Notebook

Paso 9: Conectar las Rutas (URLs)



El archivo `urls.py` es el despachador de tráfico. Mapea la dirección web exacta que el usuario visita con la función de la Vista que acabamos de construir.

misitio.com/tareas/

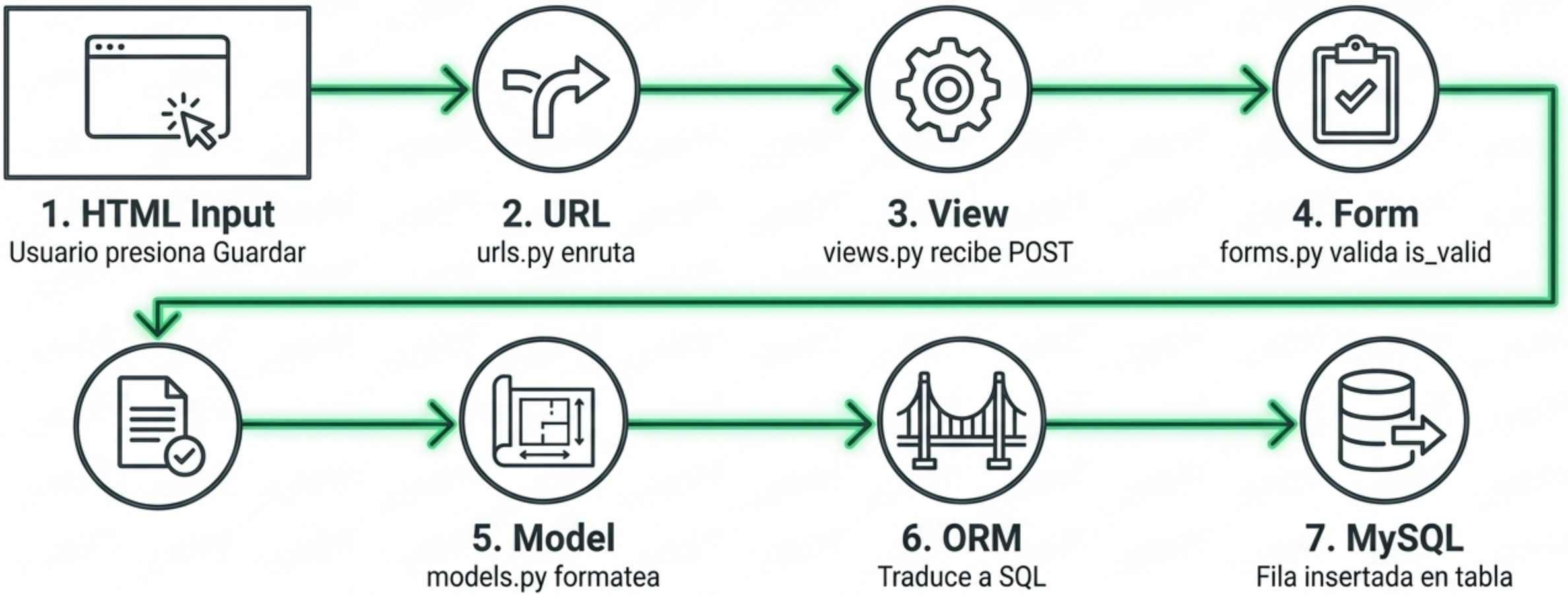
```
from django.urls import path
from . import views

urlpatterns = [
    path('tareas/', views.task_list, name='task_list'),
]
```

Gemini Notebook

Síntesis: El Flujo Completo MVT

Una visión integral de cómo una solicitud viaja desde un clic en el navegador hasta convertirse en un registro permanente en la base de datos MySQL.



Gemini Notebook

Lo que el ORM hizo por ti

Una sola línea de código en Python abstrae docenas de líneas de código SQL, previniendo inyecciones SQL y gestionando las conexiones de manera segura y eficiente.

El Método Django (Simple y Seguro)

```
form.save()
```



VS

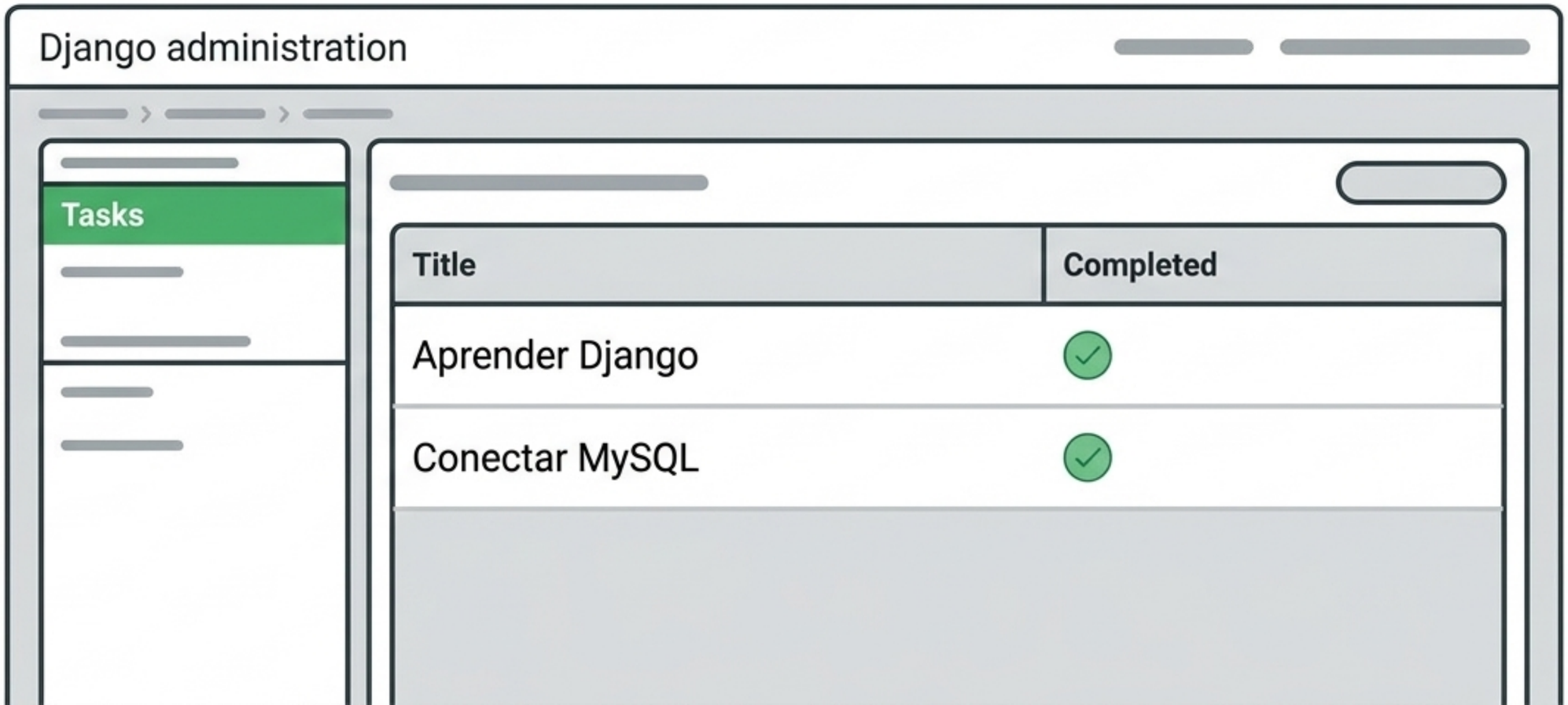
SQL Crudo (Complejo y Propenso a Errores)

```
cursor = connection.cursor()
sql = '''INSERT INTO app_task
(title, description, completed)
VALUES (%s, %s, %s)'''
cursor.execute(sql, [title, desc,
completed])
connection.commit()
cursor.close()
```

Gemini Notebook

Verificando los Datos: Django Admin

No necesitas entrar a la consola de MySQL para comprobar tus datos. Registrando el modelo en admin.py, Django proporciona una interfaz gráfica completa lista para usar, que lee directamente desde tu base de datos MySQL.



Gemini Notebook

Resumen de Comandos Esenciales

```
$ django-admin startapp
```

Crea la estructura de carpetas (MVT) para una nueva aplicación.

```
$ python manage.py makemigrations
```

Traduce tus modelos Python en planos de base de datos.

```
$ python manage.py migrate
```

Ejecuta los planos y crea las tablas físicas en MySQL.

```
$ python manage.py runserver
```

Inicia el servidor local para probar la interfaz web.

Gemini Notebook

Construido para Escalar

La arquitectura MVT de Django y el poder del ORM **transforman** la complejidad de MySQL en bloques de **construcción manejables.** Al separar la lógica, los datos y la presentación, has **creado un sistema limpio, seguro y preparado para crecer.**

Gemini Notebook